applied sciences

MDPI

Article

Task Offloading Scheme Based on Proximal Policy Optimization Algorithm

Yutong Ma [1,2,*] and Junfeng Tian [1,2]

[1] Key Laboratory on High Trusted Information System in Hebei Province, Hebei University, Baoding 071000, China; tjf@hbu.edu.cn
[2] School of Cyber Security and Computer, Hebei University, Baoding 071000, China
* Correspondence: m1509336549@163.com

Abstract: The rapid development of mobile Internet technology has made users' requirements for quality of service (QoS) continuously improve. The task unloading process of mobile edge computing has the problem that it is impossible to balance delay and energy consumption for task unloading under the condition of fluctuating network bandwidth. To address this issue, this paper proposes a task offloading scheme based on the Proximal Policy Optimization (PPO) algorithm. On the basis of traditional cloud edge collaborative architecture, the collaborative computing mechanism between edge node devices is further integrated, and the concept of service caching is introduced to reduce duplicate data transmission, reduce communication latency and network load, and improve overall system performance. Firstly, this article constructs an energy efficiency function with a certain weight ratio of energy consumption and latency as the core optimization objective. Then, the task offloading process of mobile terminal devices is modeled as a Markov Decision Process (MDP). Finally, the deep reinforcement learning PPO algorithm is used for training and learning, and the model is solved. The simulation results show that the proposed scheme has significant advantages in reducing energy consumption and latency compared to the comparative scheme.

Keywords: mobile edge computing; computing offload; deep reinforcement learning; service cache

check for updates

Academic Editor: Jose María Alvarez Rodríguez

Received: 9 April 2025
Revised: 23 April 2025
Accepted: 23 April 2025
Published: 25 April 2025

Citation: Ma, Y.; Tian, J. Task Offloading Scheme Based on Proximal Policy Optimization Algorithm. *Appl. Sci.* **2025**, *15*, 4761. https://doi.org/10.3390/app15094761

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

With the rapid development of mobile Internet and intelligent devices, mobile communication technology has become a kind of infrastructure closely related to people's daily life, which is widely used in social networking, online shopping, entertainment, and many other fields. At the same time, the data and computation required by mobile terminal devices are also showing an exponential growth trend, which has brought tremendous pressure to mobile networks. The limited computing and communication capabilities make it impossible for mobile terminal devices to meet the demands of massive computing tasks. Mobile Cloud Computing (MCC) [1], as an emerging computing model utilizing the high-performance computing and storage capabilities of the cloud to provide computer resources and storage services for mobile terminal devices, has become an effective solution to the problem. However, in recent years, applications such as augmented reality and autonomous driving that require a large amount of computation and bandwidth have emerged. The traditional mobile cloud computing model has significant transmission delays during task processing, which cannot meet the latency-sensitive requirements of

real-time tasks. The main reason is that the distance between mobile terminal devices and central cloud and edge node devices is relatively long [2].

In order to solve the above problems, the new concept of mobile edge computing (MEC) was proposed [3,4]. This computing method reduces the distance between users and remote ends, reduces task processing delay, and improves the efficiency of task processing by distributing the original centralized computing resources and storage resources on edge node devices close to mobile terminal users. Through this approach, the problem of significant transmission latency in traditional cloud computing models is solved, which has a significant impact on improving user experience. MEC technology allocates computing resources reasonably based on dynamic changes in the network, ensuring service quality while meeting the different latency and energy consumption requirements of different user tasks [5].

Despite the rapid development of MEC technology in multiple fields, MEC still faces many challenges as network size and task complexity continue to increase. When faced with complex computing tasks or excessive workload, it is still necessary to offload the tasks to the central cloud for processing. Therefore, it is necessary to coordinate the processing of computing tasks among the central cloud, edge node devices, and mobile terminal devices [6]. The cloud-side collaboration mechanism can give full play to the powerful computing power of cloud computing and the low-latency advantage of edge computing, but it also needs to consider the limited resources and heterogeneous services of mobile edge servers [7]. Due to the limited computing and storage resources of edge node devices, only some services can be cached, and the caching strategy of the services can affect users' decisions on computing offloading. In addition, uploading pending tasks to the cloud for processing carries the risk of private data leakage during channel transmission [8]. Therefore, optimizing the task offloading strategy under the cloud edge collaboration mechanism while ensuring user data privacy, as well as achieving the comprehensive optimization of latency and energy consumption as much as possible, is a challenging and significant problem.

The concept of task offloading is very important in mobile edge computing networks. It reduces the latency of processing tasks, improves response time, and enhances user experience, and it plays an important role in achieving load balancing, reducing device energy consumption, and improving resource utilization. The main contributions of this article include the following:

(1) Building a complete mobile edge computing offload network architecture: Based on the traditional cloud edge collaborative network architecture, we integrate the edge collaborative mechanism, introduce the concept of service cache, reduce duplicate data transmission, reduce communication delay and network load, and achieve efficient data transmission and processing.
(2) Optimizing system energy consumption and delay balance: This article models task offloading and resource allocation as Markov Decision Processes and uses deep reinforcement learning algorithms to solve these problems through near-end policy optimization, thereby achieving the optimal overall energy efficiency of the system.

2. Related Work

One of the main goals of MEC networks is to offload computing tasks to network edge servers for processing, reduce the computing load on terminal devices, and thus improve application performance and system efficiency. Edge computing devices have certain computing, storage, and communication capabilities, which can provide effective computing support for mobile devices. Therefore, task offloading has become a core component of MEC. Task offloading can alleviate the computing pressure on terminal devices, reduce network load, optimize bandwidth utilization, and provide users with low-latency computing services [9]. The goals of task offloading decisions usually have

three aspects: reducing latency, reducing energy consumption, and weighing the pros and cons between latency and energy consumption [10].

In terms of reducing latency, Reference [11] solved the problem of the low processing capacity of edge servers by offloading some tasks of certain mobile terminal users to the central cloud for computing. This effectively reduced the processing delay of tasks, thus avoiding network congestion. Reference [12] attached great importance to the peak aggregation of different edge servers, extensively explored the trade-off between communication delay and computation delay in task offloading, and proposed a hierarchical task offloading and resource optimization mechanism based on tree hierarchy. Reasonably scheduling tasks based on the processing capacity of each server significantly improved network resource utilization. Reference [13] studied the task offloading scheme for power terminals with limited computer resources in the smart grid environment. This scheme used an enhanced adaptive genetic algorithm to solve the optimal task offloading strategy. Reference [14] proposed the concept of end-to-end collaboration based on cloud–edge collaboration. Although the experimental results showed that the total delay of task execution could be reduced, the computation of the task scheduling (CTS) algorithm in this article was relatively complex. Reference [15] used Markov decision theory to solve the problem of minimizing delay. Reference [16] established a WPMECN network model based on deep reinforcement learning, which has significant advantages in CPU processing latency.

In terms of reducing energy consumption, Reference [17] proposed a home energy management system based on artificial neural networks, which significantly reduces household electricity costs and carbon dioxide emissions by integrating solar photovoltaic and maximum power point tracking (MPPT) technologies. This research showed how to optimize household energy use through intelligent algorithms and provides a useful reference for energy management in mobile edge computing. In the MEC environment, similar intelligent energy management strategies can be used to optimize the energy consumption of edge devices. Reference [18] proposed a new adaptive particle swarm optimization algorithm, which has better energy-saving effects. Reference [19] focused on the issues of power control and computing resource allocation and proposed the E2PC (Energy-saving Power Control) algorithm. Although this algorithm has significant advantages in reducing energy consumption, it is difficult to apply in practice.

In terms of balancing delay and energy consumption, Reference [20] proposed a joint optimization strategy to achieve the dual objectives of energy consumption and delay optimization, which is of great significance for promoting energy efficiency improvement in MEC applications. Reference [21] proposed an optimization framework for computing offloading and resource allocation to improve network computing efficiency and save energy. Reference [22] investigated the offloading decision problem of MEC networks in multi-layer 6G network environments. The service needs of users, network status, and computing resources were fully considered to optimize network performance and user experience. Reference [23] provided a framework with an alternating-direction multiplier method to optimize the decision-making process of task offloading in a way that optimizes resource utilization and system performance. By synchronously offloading computing tasks at three levels, terminal devices, edge servers, and cloud servers, this framework minimizes the computational burden on the network, reduces network latency, and improves energy efficiency to the greatest extent possible.

3. Preliminary

3.1. Markov Decision Process Theory

The Markov Decision Process (MDP) is constructed based on Markov theory and is used for optimal decision-making in dynamic environments [24]. The core is Markovian,

which means that the future state is only determined by the current state and has nothing to do with history. The MDP can use methods such as dynamic programming to find the optimal strategy, thereby maximizing the cumulative reward.

MDP is generally represented by a five-part tuple (S, A, P, R, γ):

S represents the state space; it represents the set of all possible states in the system.

A represents the action space; it defines the set of actions that can be executed in each state.

P represents the state transition function; it refers to the probability that the system will transition to the next state given a specific current state, d, and action.

R stands for the reward function; it is used to measure the immediate reward or punishment obtained after performing a specific action in a certain state.

γ denotes the discount factor, which assesses the importance of future rewards to current decisions, and generally takes values within [0, 1].

In practical applications, the MDP is widely used in fields such as artificial intelligence decision-making, resource allocation, and path planning, providing strong modeling tools. Common solutions include value iteration, strategy iteration, and deep reinforcement learning (DRL). During the interaction with the environment, the system observes the state, selects actions, obtains rewards, and continuously optimizes its decision-making strategy.

The decision-making process of the MDP has discreteness. In each time step, the system will select an action based on the current state. The environment will transition from the current state to the next state based on the probability of state transition and then provide feedback rewards. This process will continue until it reaches the termination state or the task is completed.

3.2. Deep Reinforcement Learning Theory

Reinforcement learning is a machine learning method that is related to deep learning, as shown in Figure 1. It learns by interacting with the environment and does not require a large amount of training data [25]. The agent decides actions based on reward signals and continuously explores and learns to maximize accumulated rewards. Deep reinforcement learning (DRL) combines the techniques of deep learning and reinforcement learning [26], using deep neural networks to represent decision strategies and training them with reinforcement learning algorithms. DRL can handle high-dimensional inputs and outputs and adapt to complex environments. Common methods include the Deep Q-Network (DQN) [27], DDPG [28], and Proximal Policy Optimization (PPO) [29].

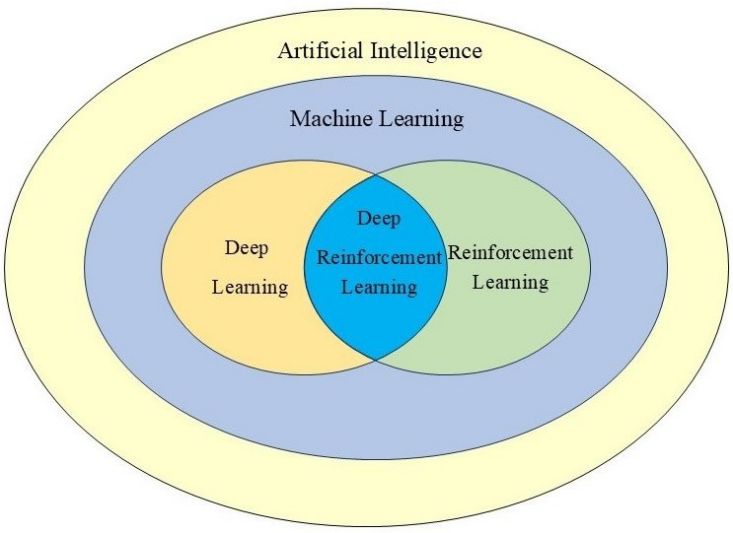

Figure 1. Deep reinforcement learning relationship.

4. Model System

Edge computing can reduce transmission delay, enhance privacy security, and reduce cloud load by sinking computing resources. However, limited by hardware resources and power consumption, edge devices make it difficult to independently complete computationally intensive tasks, and a single task offloading strategy cannot fully optimize computing efficiency, energy consumption, and resource utilization. Therefore, how to allocate computing tasks reasonably between the cloud and the edge and achieve a balance between computing efficiency, energy consumption control, and latency optimization has become a research hotspot.

This article proposes a task offloading scheme that utilizes the Proximal Policy Optimization (PPO) algorithm. The basic framework of this method is shown in Figure 2. In the traditional cloud edge collaboration architecture, the collaboration mechanism between edges is introduced to improve the utilization of computing resources by optimizing task offloading paths and resource allocation. Simultaneously, service caching technology is utilized to effectively reduce data transmission latency and network load.

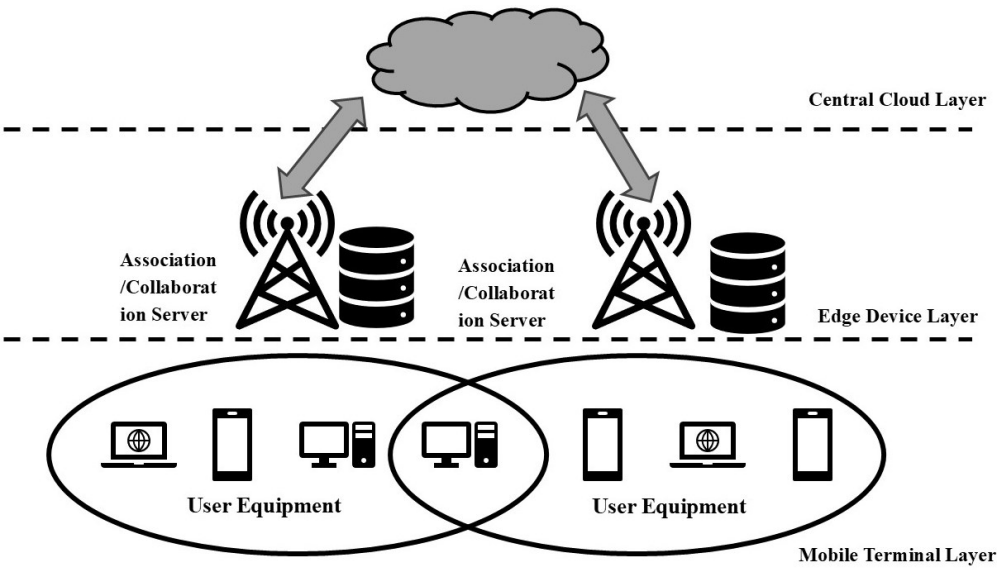

Figure 2. System model diagram of mobile edge computing task unloading.

4.1. Model

As shown in Figure 2, this chapter examines an edge computing task unloading network composed of multiple edge computing nodes and mobile terminal devices within their coverage areas. The network is connected to a cloud computing center to explore efficient task unloading strategies in a mobile edge computing environment. The network architecture is primarily divided into three layers: the Central Cloud Layer (CCL), the Edge Device Layer (EDL), and the Mobile Terminal Layer (MTL). The MTL consists of a large number of mobile terminal devices, including smartphones, tablets, Internet of Things (IoT) devices, and sensors, denoted as $\mathcal{N} = \{1, 2, \ldots, N\}$, where n represents the device number, with $1 \leq n \leq N$. These terminal devices interact with users and are responsible for generating computing tasks. The EDL comprises multiple edge computing nodes, denoted as $\mathcal{M} = \{1, 2, \ldots, M\}$, where m represents the edge server number, with $1 \leq m \leq M$. Due to the dense deployment of edge devices and overlapping coverage areas, end users located in overlapping areas choose the edge devices with the best channel conditions for association.

4.2. Service Cache Model

We assume that the edge computing nodes in this chapter require the local caching of corresponding services to provide services to mobile terminal users. However, due to the needs for data security and privacy protection, load balancing constraints, and the limitations of storage space on edge computing nodes, a single edge computing node cannot store all types of service caches. $\mathcal{S}$ is a set of service caches, which includes S different types of service cache resources. The data size of each cache resource is represented by D_s, where $s = \{1, 2, \ldots, S\}$. The deployment status of service caches on edge computing nodes can be represented by the variable $\mathbf{C} = \{C_s\}_{s=1}^{S}$, where C_s is a Boolean variable. When the s-th type of service cache is deployed on the edge computing node, $C_s = 1$; otherwise, $C_s = 0$. Tasks generated by mobile terminal devices depend on a specific type of service cache, and the edge computing node can only correctly execute the corresponding computing tasks if it has pre-deployed the service cache resources required by the tasks. If the associated edge computing node of the mobile terminal device lacks the service cache resources needed for task execution, the task needs to be migrated to other edge computing nodes that have the required cache resources to execute. If the edge computing nodes also lack the necessary resources, the service cache must be downloaded from the central cloud.

4.3. Task Communication Model

In this article, edge node devices use Time Division Multiple Access (TDMA) to communicate with mobile terminal users. Edge node devices allocate different time slots for each associated mobile terminal user to transmit data, ensuring that only one user communicates on a specific channel at a time. Therefore, data transmission between different users will not conflict, effectively avoiding interference between channels and improving the communication efficiency and stability of the system.

(1) Communication Model between Edge Nodes and Mobile Terminals

According to Shannon's theorem, in an interference-free wireless communication calculation model, assuming that the channel transmission gain is g, and assuming that mobile terminal users do not move within a single time slot, then g is a constant value. The transmission power is $P_n^{tran,d}$, the noise power spectral density is N_0, and the wireless channel bandwidth is B_t^m. From this, the uplink channel transmission rate r_{nm} from the terminal device n to the edge node m can be obtained as follows:

$$r_{nm} = B_t^m \times \log_2\left(1 + \frac{P_n^{tran,d} \times g}{N_0}\right), \forall n \in \mathcal{N}, m \in \mathcal{M}, t \in T \tag{1}$$

The transmission delay $T_{n,m}^{tran,d}$ for the terminal device n to offload a task of the data size D_n to the edge node m is

$$T_{n,m}^{tran,d} = \frac{D_n}{r_{nm}} = \frac{D_n}{B_t^m \times \log_2\left(1 + \frac{P_n^{tran,d} \times g}{N_0}\right)}, \forall n \in \mathcal{N}, m \in \mathcal{M}, t \in T \tag{2}$$

Given that the scale of task results is much smaller than that of the input data, and the uplink transmission rate from end users to edge devices is significantly lower than the downlink transmission rate from edge devices to end users, in the communication model between end users and edge devices, only the uplink is focused on, while ignoring the delay overhead generated when edge devices transmit task results back to end users.

(2) Communication Model between Edge Nodes

Communication between multiple edge nodes is similar to the communication process between edge nodes and mobile terminal users, and the transmission also requires the use of a channel resource. This is represented as $r_{m_1 m_2}$:

$$r_{m_1 m_2} = B_t^m \times \log_2\left(1 + \frac{P_{m_1}^{tran,e} \times g}{N_0}\right), m \in \mathcal{M}, t \in T \tag{3}$$

When an edge node, m_1, lacks the service cache type required for task execution, and it is necessary to offload the task to another edge node, m_2, for execution, the migration delay between m_1 and m_2 can be expressed as

$$T_{m_1,m_2}^{tran,e} = \frac{D_n}{r_{m_1 m_2}}, \forall n \in \mathcal{N}, m \in \mathcal{M}, t \in T \tag{4}$$

(3) Communication Model between Edge Nodes and the Central Cloud

When both the associated edge node m_1 and the cooperative edge node m_2 do not contain the service cache type required for task execution, it is necessary to download the required service cache resources from the central cloud. The transmission delay for the edge node m to download service cache from the central cloud through a return path is

$$T_m^{tran,e} = \frac{D_s}{W_m}, \forall s \in \mathcal{S}, m \in \mathcal{M} \tag{5}$$

where W_m represents the bandwidth of the return path.

4.4. Energy Consumption Model

(1) Task Upload Energy Consumption

The energy consumption of mobile terminal devices uploading tasks to their associated edge computing nodes for processing is expressed as

$$E_{n,m}^{tran,d} = P_n^{tran,d} T_{n,m}^{tran,d}, \forall n \in \mathcal{N}, m \in \mathcal{M} \tag{6}$$

where $P_n^{tran,d}$ is the transmission power of the terminal device n.

(2) Task Computing Energy Consumption

The computing delay for mobile terminal devices uploading tasks to edge computing nodes is

$$T_{n,m}^{comp,e} = \frac{D_n \times C_m}{f_m}, \forall n \in \mathcal{N}, m \in \mathcal{M} \tag{7}$$

where f_m represents the CPU frequency of the edge node.

The computing energy consumption for mobile terminal devices uploading tasks to edge computing nodes is

$$E_{n,m}^{comp,e} = P_m^{comp,e} T_{n,m}^{comp,e}, \forall n \in \mathcal{N}, m \in \mathcal{M} \tag{8}$$

where $P_m^{comp,e}$ represents the computing power of edge nodes.

(3) Task Migration Energy Consumption

The migration energy consumption for offloading tasks between the associated edge node m_1 and the cooperative edge node m_2 is

$$E_{m_1,m_2}^{tran,e} = P_m^{tran,e} T_{m_1,m_2}^{tran,e}, \forall m \in \mathcal{M} \tag{9}$$

(4) Service Cache Transmission Energy Consumption

The transmission energy consumption for the edge node m to download service cache from the central cloud is

$$E_c^{tran,e} = P_c^{tran,c} T_m^{tran,e}, \forall m \in \mathcal{M} \tag{10}$$

4.5. Task Offloading Model

The time period T for the task offloading of terminal devices is divided into several small time segments, t, assuming that the user's location does not change during each small time segment, t. In the time segment t, the service request of the terminal device n is

$$j_n(t) = (F_n(t), S_n(t), D_n(t)), t \in T \tag{11}$$

These, respectively, represent the number of CPU cycles required for user-requested tasks, the service cache type on which task execution depends, and the data volume of user tasks.

As shown in Figure 3, within each time slot, each computation task generated by the terminal user is offloaded to the edge node device with the best channel quality associated with it. If the edge device associated with the terminal user lacks the service cache required for task execution, then the edge device will further offload the computation task to other collaborative edge node devices or download the required service cache from the central cloud.

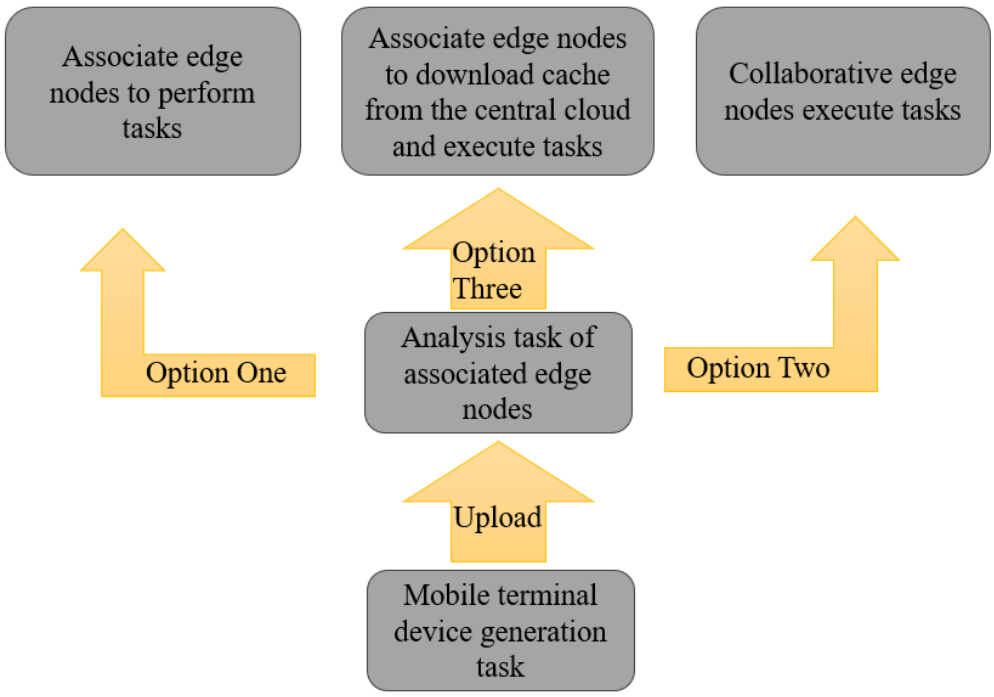

Figure 3. Schematic diagram of mobile edge computing task unloading process.

4.5.1. Associated Edge Nodes Perform Tasks

If the mobile terminal user offloads the task to its associated edge node and this edge node has service cache, then the execution delay consists of two parts: first, the transmission delay for the mobile terminal user to send the task to the edge node; second, the computation delay for the edge node to execute the task.

$$T_{ch1} = T_{n,m}^{tran,d} + T_{n,m}^{comp,e} \tag{12}$$

The energy consumption corresponding to the two operation delays is

$$E_{ch1} = E_{n,m}^{tran,d} + E_{n,m}^{comp,e} \tag{13}$$

If the mobile terminal user offloads the task to its associated edge node and this edge node does not have service cache, then the execution delay consists of three parts: (1) the transmission delay for the mobile terminal user to upload the task to the edge node; (2) the transmission delay for the associated edge node to download service cache from the central cloud; (3) the computation delay for the associated edge node to execute the task.

$$T_{ch3} = T_{n,m}^{tran,d} + T_{n,m}^{comp,e} + T_{m}^{tran,e} \tag{14}$$

The energy consumption corresponding to the three operation delays is

$$E_{ch3} = E_{n,m}^{tran,d} + E_{n,m}^{comp,e} + E_{c}^{tran,e} \tag{15}$$

4.5.2. Collaborative Edge Nodes Execute Tasks

If the mobile terminal user offloads the task to a cooperative edge node, then the execution delay includes three parts: (1) the transmission delay for the mobile terminal user to upload the task to the edge node; (2) the migration delay for transferring data between the edge nodes m_1 and m_2; (3) the computation delay for the cooperative edge node to execute the task.

$$T_{ch2} = T_{n,m}^{tran,d} + T_{m_1,m_2}^{tran,e} + T_{n,m}^{comp,e} \tag{16}$$

The energy consumption corresponding to the three operation delays is

$$E_{ch2} = E_{n,m}^{tran,d} + E_{m_1,m_2}^{tran,e} + E_{n,m}^{comp,e} \tag{17}$$

4.6. Problem Description

The objective of this chapter is to minimize the user's delay under the consideration of the delay constraints of computing tasks, enhance the quality of service for users, and ensure the minimization of energy consumption in cloud computing. Based on ensuring service quality and constraint conditions, system energy consumption should be reduced and green energy-efficient computing should be achieved. The problem is formulated as follows:

$$P: \min_{\alpha,\beta,\mathcal{N}\in\mathcal{N}} \sum_{n=1}^{N}\sum_{m=1}^{M}\sum_{t=1}^{T}(\alpha \cdot T_{n,m,t} + \beta \cdot E_{n,m,t}) \tag{18}$$

subject to:

$$\begin{aligned}
&\text{C1:} \quad \alpha + \beta = 1 \\
&\text{C2:} \quad \alpha \in [0,1], \beta \in [0,1] \\
&\text{C3:} \quad T_n \le T_n^{\max} \\
&\text{C4:} \quad \sum_{m=1}^{M} p_m = 1 \\
&\text{C5:} \quad \sum_{m=1}^{M} q_m = 1
\end{aligned}$$

4.7. Problem Solving

4.7.1. Markov Decision Process

This paper models the offloading process of edge computing tasks as a Markov Decision Process (MDP). The core of the MDP lies in the fact that current decisions are based solely on the current state, independent of previous states; future decisions only need to consider the changes in the environment after the current decision, without being affected by previous decisions. Each choice made by the mobile terminal device is based on the current environmental state, without relying on past decisions and states, to determine the action strategy. The MDP typically includes three parts—the state space, action space, and reward function—with the ultimate goal being to select the optimal action strategy to maximize cumulative rewards.

4.7.2. State Space

During the time period T for task offloading on edge devices, before a certain time segment, t, starts, the system's state space can be represented as $S_t = [j_n(t), x_n(t), c_m(t), B_m(t)]$, where $j_n(t)$ indicates the current task generation and upload status of the terminal device, $x_n(t)$ represents the current user's location information, $c_m(t)$ represents the service cache information in the edge computing device, and $B_m(t)$ represents the current bandwidth situation of the edge computing device.

4.7.3. Action Space

The actions taken by the intelligent agent for decision-making are represented as

$$AC = \{p, q\} \tag{19}$$

where p represents the user's choice of an edge cloud server to upload and execute the computing task, and q represents the cooperative server selected during the task offloading process. The appropriate offloading strategy is chosen based on the service cache status in the servers to ultimately execute the task.

4.7.4. Reward Function

To reduce the system's delay and energy consumption during operation, a reasonable offloading strategy must be formulated, which means that the design of the reward function must align with the system's optimization goals. The purpose of reinforcement learning is to maximize the long-term cumulative reward. The reward function is the key to connecting the system's optimization goals with the reward calculation, and its expression is as follows:

$$R_t = -\sum_{n=1}^{N}\sum_{m=1}^{M}(\alpha \cdot T_{n,m,t} + \beta \cdot E_{n,m,t}) \tag{20}$$

4.7.5. Unloading Algorithm Based on Near-End Policy Optimization

PPO is an optimization algorithm suitable for problems involving discrete and continuous action spaces, used for synchronous strategy optimization. Its optimization objective involves importance sampling, but it mainly relies on the data from the previous policy, θ'. Adding KL divergence constraints in the objective function limits the difference between the behavior policy θ' and the target policy θ; thus, PPO is classified as a policy optimization algorithm. PPO evolved from the Trust Region Policy Optimization (TRPO) algorithm. The optimization process of TRPO relies on trust region methods, making it relatively complex. To address this issue, OpenAI proposed POD, which is actually a simplified version of TRPO. PPO uses a penalty term to constrain policy updates instead of directly using KL divergence constraints, adding it to the optimization objective for adjustment.

Like TRPO, PPO only adjusts the coefficients of the penalty term to ensure that policy changes are within the constraint range, thus avoiding complex dual optimization. To reduce computational complexity, PPO uses a clipping mechanism to simplify the KL divergence constraints. Compared to traditional TRPO, PPO relies on clipping policy ratios, omitting trust region optimization, which also ensures that strategy updates are not too aggressive, enhancing training stability. The objectives and algorithm of PPO optimization are shown in Formula (21) and Table 1:

$$L(\theta) \approx \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)] \tag{21}$$

Table 1. PPO algorithm steps.

Step	Description	Expression
Input	Collect $\{a_t, s_t, r_t\}$ tuples	-
Output	Update model	-
$i = 1, 2, \ldots, N_1$	Run policy θ for T times, collect $\{a_t, s_t, r_t\}$, estimate advantage $\hat{A}_t = \sum_{t'>t} \gamma^{t'-t} r_{t'} - V_\theta(s_t)$	-
$j = 1, 2, \ldots, N_2$	Optimize policy	$J^\theta_{PPO}(\theta) = J^\theta(\theta) - \beta KL(\theta, \theta')$
$k = 1, 2, \ldots, N_3$	Update value function	$L_{\text{CLIP}}(\phi) = -\sum_{t=1}^{T} \left(\sum_{t'>t} \gamma^{t'-t} r_{t'} - V_\phi(s_t) \right)^2$

5. Experiment and Performance Verification

5.1. Experimental Environment and Parameter Settings

The simulation experimental platform consisted of a device with a frequency of 2.7 GHz, 16 GB of RAM, and an Intel(R)Core(TM) i5-11400H CPU. The device runs a 64-bit Windows 10 operating system, and the simulation software was Pycharm2024.2.1.

This experiment included one central cloud server, M edge servers, and N mobile terminal devices. Each terminal device had a computation task that needed to be executed. Due to the limited computing power of mobile edge devices, all computation tasks in this chapter could only be offloaded to edge servers or the central cloud for execution. The complete parameter settings in the experiment are shown in Table 2.

Table 2. Simulation delay parameter descriptions and corresponding value settings.

Symbol	Definition	Value Range
B_t^m	Bandwidth	1–20 MHz
$P_n^{tran,d}$	Transmission power of terminal device n	0.1–2 W
$P_m^{tran,e}$	Transmission power of edge node m	1–5 W
$P_c^{tran,e}$	Transmission power of central cloud c	10–50 W
$P_m^{comp,e}$	Computing power of edge node m	10–100 W
$P_c^{comp,c}$	Computing power of central cloud	50–500 W
N_0	Noise power spectral density	10^{-9} W/Hz
g	Channel transmission gain	$2 \times 10^{-10} \sim 2 \times 10^{-9}$
C_m	Cycles per bit required by edge node m	500–2000 cycles/bit
f_c	CPU frequency of cloud server	2–3.5 GHz
f_m	CPU frequency of edge node	1–2.5 GHz
D_n	Data size of computation tasks	1 KB–1 MB
W_m	Return path bandwidth	100–1000 Mbps

5.2. Comparative Experimental Setup

To more fully verify the effectiveness of the proposed solution, the experiment compared the following four different schemes:

(1) A task offloading strategy based on the edge computing optimization algorithm (EMC-PPO).
(2) A task offloading strategy based on the edge computing optimization algorithm without edge collaboration (EMC-PPO-NO), which is also solved using the PPO algorithm. However, there is no collaboration between edge nodes, and when an edge node lacks the required service cache for task execution, it can only download the cache from the central cloud.
(3) Complete offloading to the central cloud (CLOUD).
(4) A random offloading strategy (Random).

5.3. Parameter Analysis

Figure 4 illustrates the convergence performance of the EMC-PPO method under different learning rate conditions. From Figure 4, it can be observed that when the learning rate was set to 0.001, 0.01, and 0.1, due to the learning rate being too high or too low, the adjustment steps of the model parameters were either too large or too small, making it impossible to find the optimal solution. On the other hand, when the learning rate was 0.001, the model achieved the best convergence effect and reward outcome. Therefore, 0.001 was determined to be the optimal learning rate hyperparameter.

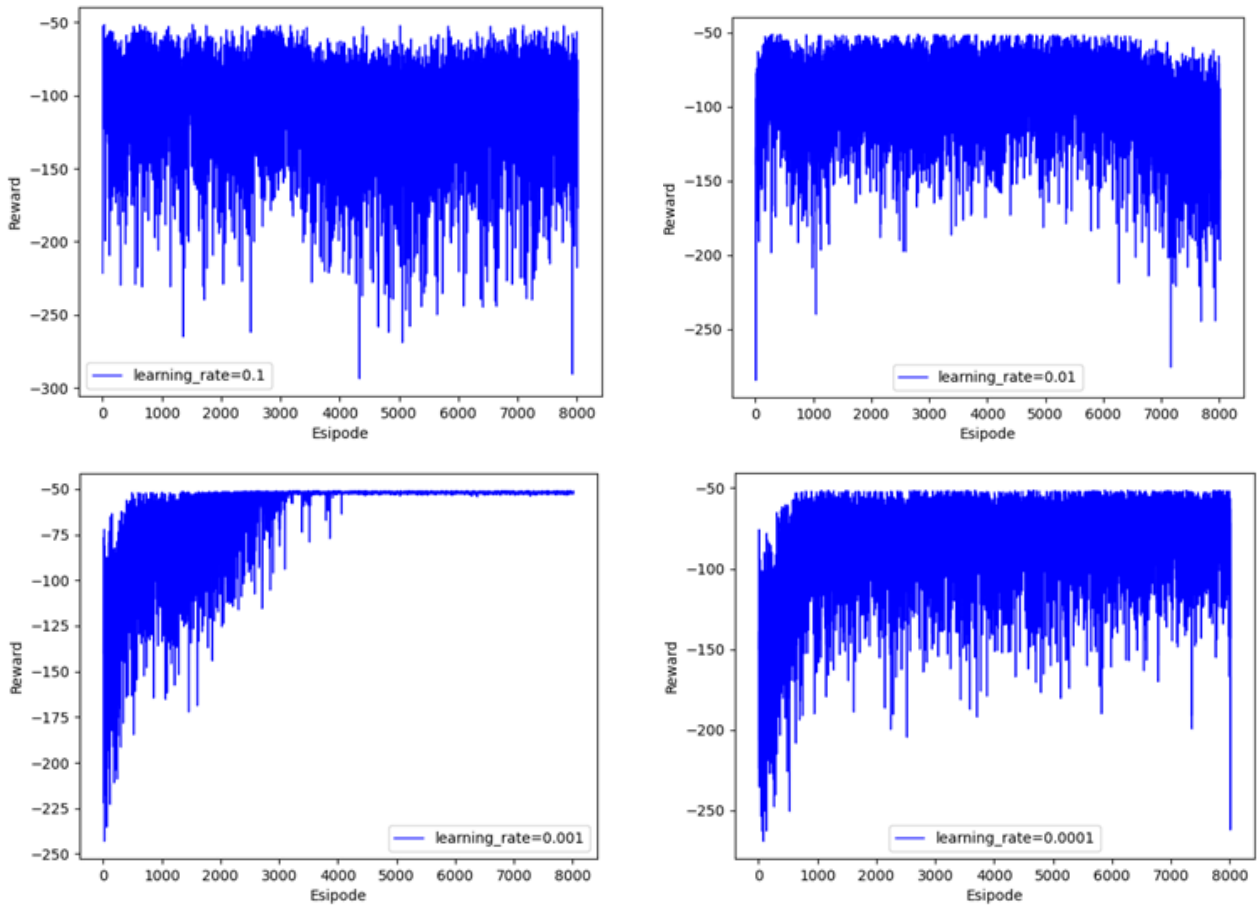

Figure 4. MEC-PPO convergence graph under different learning rates.

Figure 5 illustrates the convergence performance of the EMC-PPO method under different discount rates. From Figure 5, it can be observed that when the discount rate was set to 0.93, the model could achieve convergence in less than 4000 iterations. However, under discount rates of 0.99 and 0.90, even after 6000 iterations, the model failed to converge. Additionally, when the discount rate was 0.96, the convergence time of the model was relatively late. This paper adopted a discount rate of 0.93 as the optimal discount rate.

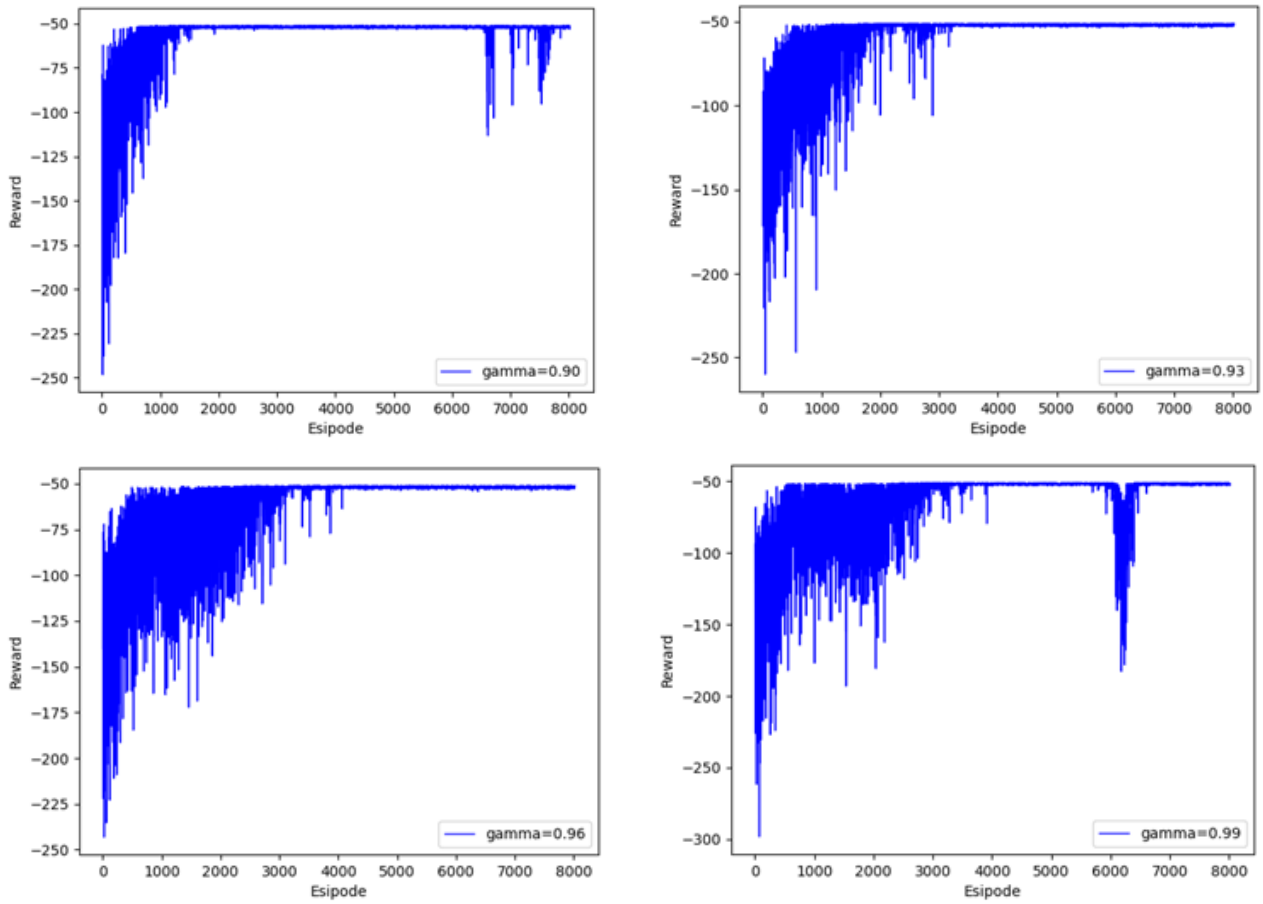

Figure 5. MEC-PPO convergence under different discounts.

5.4. Performance Comparison

In Figure 6, we compare the average total rewards of four methods: EMC-PPO, EMC-PPO-NO, CLOUD, and Random. The reward value represents the weighted sum of cloud computing energy consumption and processing delay for each event. A higher reward value implies lower energy consumption and processing delay. The EMC-PPO algorithm achieved the highest reward in the final phase and had the highest reward value in the convergence phase. Although the initial reward value of the random offloading scheme was higher than that of the EMC-PPO algorithm, it decreased as the number of training rounds increased. The final results of the EMC-PPO offloading scheme were superior. The possibility of this situation arose because the random strategy lacks the ability to adapt to changes in environmental dynamics and cannot be optimized based on environmental states and historical experience. Since the bandwidth in this paper was set to simulate real-world conditions, it fluctuated randomly. In the early training phase of the EMC-PPO algorithm, due to the lack of sufficient experience, its performance may not have been as good as the random strategy. However, it could continuously adjust the strategy based on the reward signal, gradually finding the optimal offloading strategy, thereby reducing

the total energy consumption and processing delay and ultimately achieving the highest reward value. Offloading all to the central cloud, although significantly reducing processing delay and energy consumption, increased the cost of transmission delay, resulting in poor overall performance. Without collaboration between edge clouds, tasks may have needed to be offloaded multiple times to the central cloud for processing, rather than being completed directly between edge clouds, thus increasing communication overhead and energy consumption.

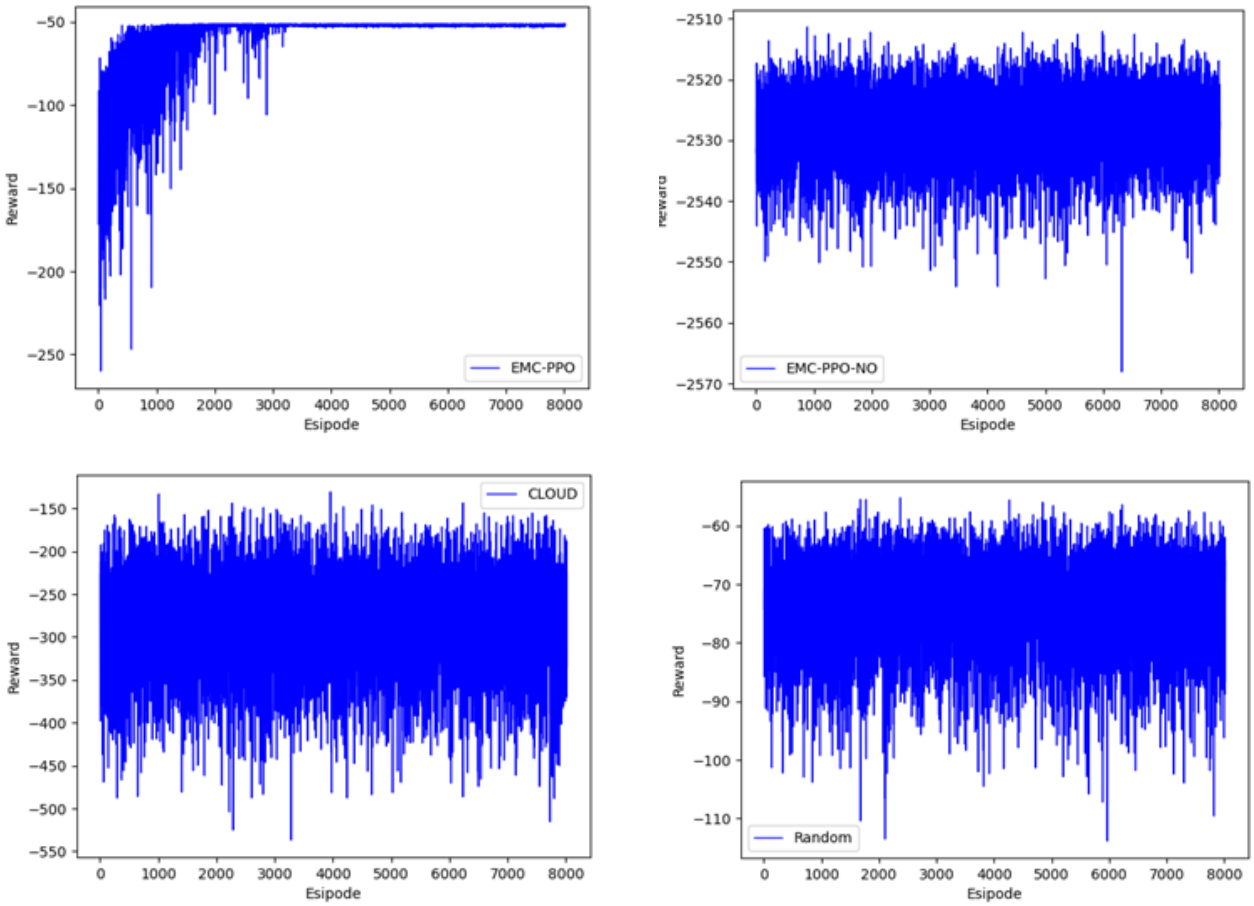

Figure 6. Reward values and convergence graphs for different uninstallation strategies

Figure 7 summarizes the reward situation under different numbers of edge computing nodes. As the number of edge devices increased, the speed at which the reward value improved accelerated. Although the rate of improvement varied, the final stable value tended to be consistent across all scenarios, stabilizing around −50. The fluctuation was relatively large in the initial stage, but as training progressed, the volatility gradually decreased and eventually stabilized. This indicates that within a certain range, increasing the number of edge nodes could enhance the system's learning efficiency by providing more space for collaboration.

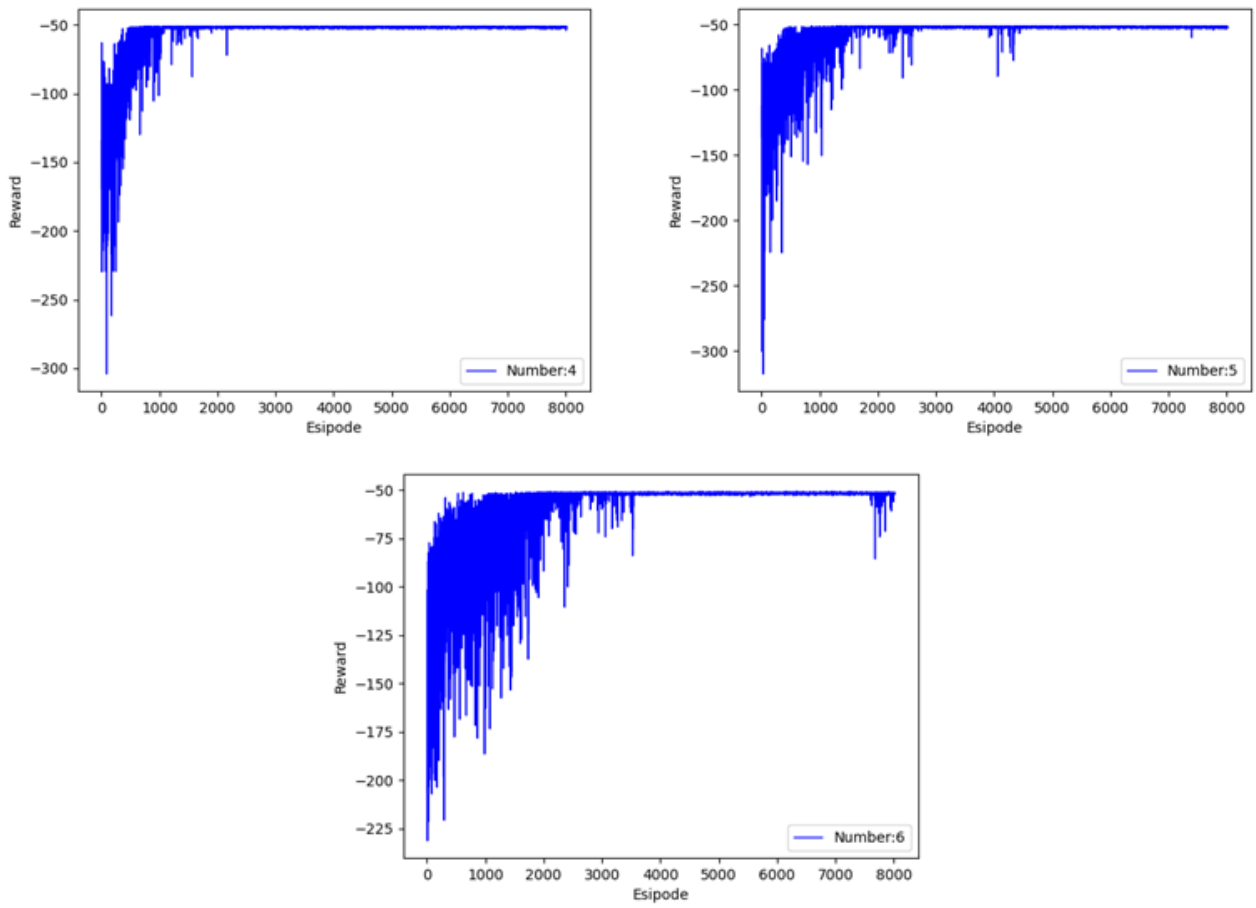

Figure 7. Graph of reward values and convergence under different numbers of edge node devices.

6. Conclusions

In this paper, the task offloading problem of mobile edge computing in MEC scenarios was studied in depth, and a task offloading scheme based on the near-end strategy optimization algorithm was proposed. This method improves task scheduling and resource allocation through collaborative computing between edge devices, thereby increasing the efficiency of computing resources. Moreover, utilizing service caching technology can avoid redundant data transmission, reduce communication latency, and network load. Next, we constructed the task offloading and resource allocation problem as a Markov Decision Process (MDP). The problem was solved using the near-end policy optimization (PPO) reinforcement learning algorithm, which can dynamically and adaptively adjust. In scenarios where the simulated network environment undergoes significant changes, the algorithm comprehensively optimizes latency, energy consumption, and computational efficiency. However, the current service caching strategy is based on static variables. In the future, we will consider incorporating cache update strategies and more intelligent algorithms to better adapt to dynamic network environments and user needs, thereby making task offloading more flexible and adaptable.

Author Contributions: Conceptualization, Y.M. and J.T.; methodology, Y.M. and J.T.; software, Y.M.; validation, Y.M.; formal analysis, Y.M. and J.T.; investigation, Y.M. and J.T.; resources, Y.M. and J.T.; data curation, Y.M.; writing—original draft preparation, Y.M.; writing—review and editing, J.T.; visualization, Y.M.; supervision, J.T.; project administration, Y.M.; funding acquisition, J.T. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Funds of the Central Government for Local Science and Technology Development (236Z0701G).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts of Interest: The authors declare that this study was conducted without any commercial or financial relationships that could be considered potential conflicts of interest.

References

1. Panigrahi, C.R.; Sarkar, J.L.; Pati, B.; Buyya, R.; Mohapatra, P.; Majumder, A. Mobile Cloud Computing and Wireless Sensor Networks: A review, integration architecture, and future directions. *Iet Netw.* **2021**, *10*, 141–161. [CrossRef]
2. Ren, J.; Zhang, D.; He, S.; Zhang, Y.; Li, T. A survey on end-edge-cloud orchestrated network computing paradigms: Transparent computing, mobile edge computing, fog computing, and cloudlet. *ACM Comput. Surv. (CSUR)* **2019**, *52*, 1–36. [CrossRef]
3. Yuan, H.; Wang, M.; Bi, J.; Shi, S.; Yang, J.; Zhang, J.; Zhou, M.; Buyya, R. Cost-efficient Task Offloading in Mobile Edge Computing with Layered Unmanned Aerial Vehicles. *IEEE Internet Things J.* **2024**, *11*, 30496–30509. [CrossRef]
4. Vilà, I.; Sallent, O.; Pérez-Romero, J. Relay-empowered beyond 5G radio access networks with edge computing capabilities. *Comput. Netw.* **2024**, *243*, 110287. [CrossRef]
5. Saleem, M.A.; Zhou, S.; Fengli, Z.; Ahmad, T.; Nigar, N.; Hadi, M.U.; Shabaz, M. Delay, Energy, and Outage Considerations in GenAI-Enhanced MEC-NOMA-Enabled Vehicular Networks. *IEEE Trans. Intell. Transp. Syst.* 2025, *Early Access*. [CrossRef]
6. Cao, K.; Hu, S.; Shi, Y.; Colombo, A.W.; Karnouskos, S.; Li, X. A survey on edge and edge-cloud computing assisted cyber-physical systems. *IEEE Trans. Ind. Inform.* **2021**, *17*, 7806–7819. [CrossRef]
7. Malazi, H.T.; Chaudhry, S.R.; Kazmi, A.; Palade, A.; Cabrera, C.; White, G.; Clarke, S. Dynamic service placement in multi-access edge computing: A systematic literature review. *IEEE Access* **2022**, *10*, 32639–32688. [CrossRef]
8. Jiang, P.; Wang, Q.; Huang, M.; Wang, C.; Li, Q.; Shen, C.; Ren, K. Building in-the-cloud network functions: Security and privacy challenges. *Proc. IEEE* **2021**, *109*, 1888–1919. [CrossRef]
9. Nain, G.; Pattanaik, K.K.; Sharma, G.K. Towards edge computing in intelligent manufacturing: Past, present and future. *J. Manuf. Syst.* **2022**, *62*, 588–611. [CrossRef]
10. Tan, L.; Kuang, Z.; Zhao, L.; Liu, A. Energy-efficient joint task offloading and resource allocation in OFDMA-based collaborative edge computing. *IEEE Trans. Wirel. Commun.* **2021**, *21*, 1960–1972. [CrossRef]
11. Sahni, Y.; Cao, J.; Yang, L. Data-aware task allocation for achieving low latency in collaborative edge computing. *IEEE Internet Things J.* **2018**, *6*, 3512–3524. [CrossRef]
12. Tong, L.; Li, Y.; Gao, W. A hierarchical edge cloud architecture for mobile computing. In Proceedings of IEEE International Conference on Computer Communications (INFOCOM), San Francisco, CA, USA, 10–14 April 2016; pp. 1–9.
13. Dou, H.; Xu, Z.; Jiang, X.; Cui, J.; Zheng, B. Mobile edge computing based task offloading and resource allocation in smart grid. In Proceedings of the 2021 13th International Conference on Wireless Communications and Signal Processing (WCSP), Changsha, China, 20–22 October 2021; pp. 1–5.
14. Lu, Y.; Zhao, Z.; Gao, Q. A distributed offloading scheme with flexible MEC resource scheduling. In Proceedings of the 2021 IEEE Smart World, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Internet of People and Smart City Innovation (Smart World/ SCALCOM/UIC/ATC/IOP/SCI), Atlanta, GA, USA, 18–21 October 2021; pp. 320–327.
15. Liu, J.; Mao, Y.; Zhang, J.; Letaief, K.B. Delay optimal computation task scheduling for mobile edge computing systems. In Proceedings of the 2016 IEEE International Symposium on Information Theory (ISIT), Barcelona, Spain, 11 August 2016; pp. 1451–1455.
16. Yu, Y.; Yan, Y.; Li, S.; Li, Z.; Wu, D. Task delay minimization in wireless powered mobile edge computing networks: A deep reinforcement learning approach. In Proceedings of the 2021 13th International Conference on Wireless Communications and Signal Processing (WCSP), Changsha, China, 20–22 October 2021; pp. 1–6.
17. Balakrishnan, R.; Geetha, V.; Kumar, M.R.; Leung, M.-F. Reduction in Residential Electricity Bill and Carbon Dioxide Emission through Renewable Energy Integration Using an Adaptive Feed-Forward Neural Network System and MPPT Technique. *Sustainability* **2023**, *15*, 14088. [CrossRef]
18. Chen, X.; Zhang, J.; Lin, B.; Chen, Z.; Wolter, K.; Min, G. Energy efficient offloading for DNN based smart IoT systems in cloud-edge environments. *IEEE Trans. Parallel Distrib. Syst.* **2022**, *33*, 683–697. [CrossRef]

19. Wu, F.; Leng, S.; Maharjan, S.; Huang, X.; Zhang, Y. Joint power control and computation offloading for energy efficient mobile edge networks. *IEEE Trans. Wirel. Commun.* **2022**, *21*, 4522–4534. [CrossRef]
20. You, C.; Huang, K.; Chae, H.; Kim, B.H. Energy-efficient resource allocation for mobile-edge computation offloading. *IEEE Trans. Wirel. Commun.* **2016**, *16*, 1397–1411. [CrossRef]
21. Wang, C.; Liang, C.; Yu, F.R.; Chen, Q.; Tang, L. Computation offloading and resource allocation in wireless cellular networks with mobile edge computing. *IEEE Trans. Wirel. Commun.* **2017**, *16*, 4924–4938. [CrossRef]
22. Rodrigues, T.K.; Liu, J.; Kato, N. Offloading decision for mobile multi-access edge computing in a multi tiered 6G network. *IEEE Trans. Emerg. Top. Comput.* **2021**, *10*, 1414–1427. [CrossRef]
23. Wang, Y.; Tao, X.; Zhang, X.; Zhang, P.; Hou, Y.T. Cooperative task offloading in three-tier mobile computing networks: An ADMM framework. *IEEE Trans. Veh. Technol.* **2019**, *68*, 2763–2776. [CrossRef]
24. Puterman, M.L. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*; John Wiley & Sons: Hoboken, NJ, USA, 2014.
25. Li, Y. Deep reinforcement learning: An overview. *arXiv* **2017**, arXiv:1701.07274.
26. Mosavi, A.; Faghan, Y.; Ghamisi, P.; Duan, P.; Ardabili, S.F.; Salwana, E.; Band, S.S. Comprehensive review of deep reinforcement learning methods and applications in economics. *Mathematics* **2020**, *8*, 1640. [CrossRef]
27. Osb, I.; Blundell, C.; Pritzel, A.; Van Roy, B. Deep exploration via bootstrapped DQN. In Proceedings of the 30th Conference on Neural Information Processing Systems (NIPS 2016), Barcelona, Spain, 5–10 December 2016.
28. Hou, Y.; Liu, L.; Wei, Q.; Xu, X.; Chen, C. A novel DDPG method with prioritized experience replay. In Proceedings of the 2017 IEEE International Conference on Systems, Man, and Cybernetics (SMC), Banff, AB, Canada, 5–8 October 2017; pp. 316–321.
29. Huang, S.; Kanervisto, A.; Raffin, A.; Wang, W.; Ontañón, S.; Dossa, R.F.J. A2C is a special case of PPO. *arXiv* **2022**, arXiv:2205.09123.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.